

CURSO 2: ALGORITMOS AVANZADOS Y ESTRUCTURAS DE DATOS

CLASE 04 • NIVEL 2 - INTERMEDIO

Clase 04: Algoritmos de Búsqueda: Lineal vs Binaria $O(\log n)$

«Búsqueda Binaria como Buscar una Palabra en el Diccionario Dividiendo a la Mitad»

Guía de estudio oficial y manual técnico. Diseñado para dominar la programación en Python mediante modelos mentales rigurosos, diagramas de flujo y código de producción.

Autor & Mentor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Python: 3.10+ | **Licencia:** MIT | **wisrovi SUITE**

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Presentación de la Sesión

Bienvenido/a a la **CLASE 04** del **Curso 2: Algoritmos Avanzados y Estructuras de Datos**. Esta guía técnica está estructurada para proporcionarte las bases conceptuales, arquitectónicas y prácticas indispensables para tu progreso.

Objetivos de la Sesión

Competencia Conceptual: Comprender la técnica 'Divide y Vencerás' y la reducción logarítmica del espacio de búsqueda.

Competencia Práctica: Implementar búsqueda binaria iterativa y usar el módulo estándar bisect.

Estructura del Documento

Sección	Contenido Temático	Objetivo Pedagógico
Pág 4: Fundamento	Teoría, Modelo Mental y Metáfora	Comprensión del concepto
Pág 5: Flujo	Diagrama de Arquitectura de Memoria	Visualización del flujo interno
Pág 6: Código	Implementación en Python 3.10+	Patrones idiomáticos (PEP 8)
Pág 7: Gotchas	Antipatrones y Trampas Comunes	Prevención de bugs en producción
Pág 8: Conclusiones	Cierre de Sesión & Notas de Repaso	Consolidación del aprendizaje
Pág 9: Bibliografía	Fuentes Canónicas y Desafío	Ampliación y autoestudio

Clase 04: Algoritmos de Búsqueda: Lineal vs Binaria $O(\log n)$

Buscar elementos en grandes volúmenes de datos requiere algoritmos más inteligentes que la simple inspección secuencial.

☀ Metáfora Central: Búsqueda Binaria como Buscar una Palabra en el Diccionario Dividiendo a la Mitad

Para encontrar la página 500 de un libro de 1000 páginas, lo abres a la mitad exacta y descartas 500 páginas de golpe.

Principios Teóricos y Modelo Mental

Búsqueda Lineal $O(n)$: Inspecciona uno a uno. Funciona en listas desordenadas.

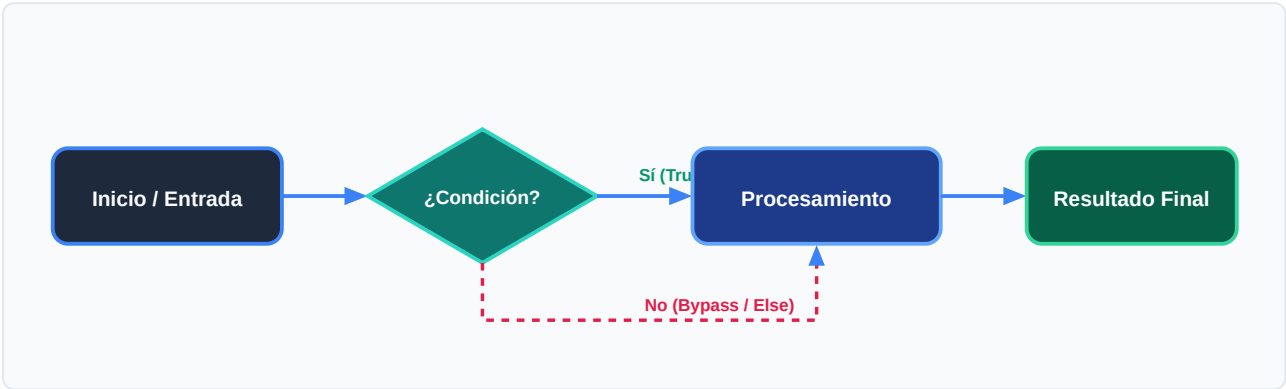
Búsqueda Binaria $O(\log n)$: Requiere que la lista esté estrictamente ordenada. En 1 millón de elementos tarda máximo 20 pasos.

⚡ Regla de Oro en Python

La búsqueda binaria solo es válida sobre arreglos previamente ordenados.

Arquitectura y Movimiento de Datos en Memoria

Reducción del espacio de búsqueda mediante punteros left, right y mid.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Inicialización de punteros: $left = 0$, $right = len - 1$.	Ventana completa activa.
2. Evaluación	Cálculo del punto medio: $mid = (left + right) // 2$.	Puntero mid evaluado.
3. Transformación	Comparación: $arr[mid]$ vs target.	Descarte de la mitad izquierda o derecha.
4. Retorno / Salida	Ajuste de límites o retorno del índice.	Elemento encontrado o -1.

Visualización Mental

Cada paso divide el problema exactamente por 2.

Código de Demostración en Python 3.10+

Implementación de Búsqueda Binaria pura y módulo bisect:

main.py (Python 3.10+)

PEP 8 Compliant

```
import bisect

def busqueda_binaria(arr: list[int], target: int) -> int:
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

datos = [10, 20, 30, 40, 50, 60, 70, 80]
idx = busqueda_binaria(datos, 60)
print("Índice de 60:", idx) # 5
print("Índice con bisect_left:", bisect.bisect_left(datos, 60))
```

Análisis de Ingeniería del Código

Punteros enteros ajustados en cada iteración garantizando terminación en $O(\log n)$.



Buena Práctica de Tipado

El uso sistemático de Type Hints (PEP 484) permite a los editores modernos como VS Code activar autocompletado inteligente y detectar errores antes de la ejecución.

| Trampas Frecuentes de Depuración (Gotchas)

Gotchas típicos en la condición de parada.

⚠ Gotcha Frecuente (Trampa de Principiante)

Escribir `while left < right` en lugar de `left <= right` omite evaluar el último elemento restante.

Comparativa: Antipatrón vs Patrón Recomendado

❌ Antipatrón

`while left < right:` # ❌ Puede fallar si el target está en el último elemento

✅ Patrón Correcto

`while left <= right:` # ✅ Evalúa todos los casos correctamente



Consejo de Resiliencia en Producción

Usa `bisect.insort` para insertar elementos manteniendo siempre la lista ordenada.

Conclusiones Clave de la Sesión

Hemos cubierto los pilares teóricos y prácticos de **Clase 04: Algoritmos de Búsqueda: Lineal vs Binaria $O(\log n)$** . El dominio de esta unidad te proporciona una base sólida para afrontar la siguiente semana formativa.



Hitos Alcanzados

Comprensión profunda del modelo mental, capacidad de depuración de gotchas comunes y solidez en la sintaxis idiomática de Python 3.10+.

Notas de Repaso Rápido

Concepto	Punto Clave a Recordar
1. Modelo Mental	Búsqueda Binaria como Buscar una Palabra en el Diccionario Dividiendo a la Mitad
2. Regla de Oro	La búsqueda binaria solo es válida sobre arreglos previamente ordenados.
3. Gotcha Crítico	Escribir <code>while left < right</code> en lugar de <code>left <= right</code> omite evaluar el último elemento restante.
4. Buenas Prácticas	Usa <code>bisect.insort</code> para insertar elementos manteniendo siempre la lista ordenada.



Mensaje del Instructor

¡Felicitaciones por completar esta lección! Recuerda que la constancia y el pedaleo diario en tu editor de código son los que te convertirán en un programador excepcional.

Fuentes Bibliográficas Oficiales

Para profundizar en los estándares formales del lenguaje y sus implementaciones internas, consulta la documentación canónica:

Recurso Oficial	Descripción	Enlace
Documentación Python 3	Especificación canónica y biblioteca estándar	docs.python.org/3/
PEP 8 — Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones de desarrollo e ingeniería	realpython.com
Suite wisrovi en GitHub	Paquetes open source de alto rendimiento	github.com/wisrovi

🏆 Desafío Práctico para el Estudiante

🎯 Reto de la Semana

Modifica la búsqueda binaria para encontrar la primera y última posición de un elemento repetido.

🔧 Validación Automatizada

Ejecuta `pytest ejercicios/` en tu terminal de VS Code para verificar automáticamente la validez de tu solución.